

LSIS-AFS Reference Implementation - Requirements

Project Overview

Software reference implementation for LunaNet Signal-In-Space Augmented Forward Signal (LSIS-AFS) Volume A. This focuses on digital signal processing, encoding/decoding, and message generation/parsing components.

Reference Documentation

- LSIS-AFS Volume A Specification

Gateway-Based Development Approach

This project uses a **gateway-based methodology** to structure the 6-month implementation:

What is a Gateway? A gateway is an incremental milestone that delivers a complete, testable functional capability. Each gateway: - Has clear, measurable success criteria - Produces tangible outputs (code, test vectors, documentation) - Can be validated independently - Builds upon previous gateways - Represents a natural checkpoint in the development process

Why Gateways? - **Incremental Progress:** Break complex system into manageable pieces - **Early Validation:** Catch errors before they propagate - **Parallel Development:** Independent gateways can be developed concurrently - **Partial Credit:** Demonstrate value even if not all gateways complete - **Risk Management:** Identify issues early when they're easier to fix - **Team Coordination:** Clear handoff points for collaborative work

Gateway Philosophy: Validate thoroughly at each gateway before proceeding. A solid foundation enables faster progress later.

Deliverable Gateways

Gateway 0: Design & Architecture (Month 1)

Goal: Design system architecture and establish development foundation **Deliverables:** - System architecture document - Component interaction diagrams - Technology stack selection - Development environment setup - Testing strategy and framework - Code structure and module design - Interface definitions - Data format specifications

Success Criteria: - Clear architecture with defined components - Development environment ready - Testing framework established - Team aligned on approach

Gateway 1: Code Generation Library (Month 2)

Goal: Generate all spreading codes per specification **Deliverables:** - Gold code generator (PRN 1-210) - Legendre sequence generator - Weil code generator (primary and tertiary) - Secondary code generator - Validation against Annex3 reference codes - Test suite with all 210 PRN codes

Success Criteria: - All codes match reference hexadecimal values - Code generation completes in < 1 second per PRN - 100% test coverage for code generators

Gateway 2: Encoding Infrastructure (Month 3)

Goal: Implement all forward error correction codes **Deliverables:** - BCH(51,8) encoder with generator polynomial 763 - CRC-24 generator with specified polynomial - LDPC encoder (rate 1/2) for SB2, SB3, SB4 - Block interleaver (60×98) - Test vectors for each encoder

Success Criteria: - BCH encoder produces correct 52-symbol output - CRC-24 matches specification examples - LDPC encoder handles puncturing correctly - Interleaver pattern validated

Gateway 3: Message Builder (Month 3)

Goal: Build complete navigation message frames **Deliverables:** - Subframe 1 builder (FID + TOI) - Subframe 2 builder (Clock & Ephemeris) - Subframe 3 builder (Variable data) - Subframe 4 builder (Network access) - Frame assembler with sync pattern - Complete 12-second frame generator

Success Criteria: - Generate valid frames with all subframes - Correct bit allocations per specification - Frame timing matches 12-second requirement - Export frames in multiple formats

Gateway 4: Signal Generator (Month 4)

Goal: Generate baseband I/Q samples **Deliverables:** - Tiered code assembly (primary + secondary + tertiary) - BPSK modulator for AFS-I (1.023 Mchip/s) - BPSK modulator for AFS-Q (5.115 Mchip/s) - I/Q sample generator with configurable sample rate - Signal file export (binary, CSV)

Success Criteria: - Generate 12-second I/Q signal files - Code synchronization meets specification - Symbol rate: 500 symbols/s for AFS-I - I/Q samples ready for RF simulation

Gateway 5: Decoding Infrastructure (Month 5)

Goal: Decode received signals back to data **Deliverables:** - Frame synchronization (sync pattern detection) - BCH(51,8) soft-decision decoder - LDPC belief propagation decoder - CRC validator - Block deinterleaver

Success Criteria: - Detect frames with >99% reliability at SNR > 0dB - BCH decoder handles soft decisions - LDPC decoder converges in < 50 iterations - CRC validation catches errors

Gateway 6: Message Parser (Month 5)

Goal: Extract navigation data from frames **Deliverables:** - Subframe 1 parser (FID + TOI extraction) - Subframe 2 parser (Clock & Ephemeris) - Subframe 3 parser (Variable data routing) - Subframe 4 parser (Network access) - Time of Transmission calculator - Message field extractors

Success Criteria: - Parse all subframe types correctly - Extract WN, ITOW, TOI fields - Reconstruct time since LRT epoch - Handle all message types in SB3/SB4

Gateway 7: Integration & Testing (Month 6)

Goal: End-to-end validation and interoperability **Deliverables:** - Round-trip test: encode → decode → verify - Test vector suite (codes, frames, signals) - Interoperability test results with other implementations - Performance benchmarks (throughput, latency) - Compliance validation report

Success Criteria: - Round-trip recovers original data with 100% accuracy - Process 12-second frames in < 1 second - All 12 interim test codes working - Pass all LSIS compliance checks - Successful cross-implementation testing

Gateway 8: Documentation & Examples (Month 6)

Goal: Complete reference implementation **Deliverables:** - API documentation - Usage examples for all components - Tutorial notebooks - Test vector documentation - Performance analysis report

Success Criteria: - All public APIs documented - Working examples for each gateway - Clear instructions for running tests - Reproducible results

Functional Requirements

FR-1: Code Generation

The system shall generate spreading codes per LSIS specification: - FR-1.1: Gold codes (2046 chips) for PRN 1-210 - FR-1.2: Weil primary codes (10230 chips) for PRN 1-210 - FR-1.3: Secondary codes (4 chips, 4 variants) - FR-1.4: Weil tertiary codes (1500 chips) for PRN 1-210 - FR-1.5: Tiered code assembly with coherent generation

FR-2: Message Encoding

The system shall encode navigation messages: - FR-2.1: BCH(51,8) encoding for Subframe 1 - FR-2.2: LDPC(1/2) encoding for Subframes 2, 3, 4 - FR-2.3: CRC-24 generation for error detection - FR-2.4: Block interleaving (60x98) - FR-2.5: Frame assembly with sync pattern

FR-3: Signal Generation

The system shall generate baseband signals: - FR-3.1: AFS-I baseband at 1.023 Mchip/s - FR-3.2: AFS-Q baseband at 5.115 Mchip/s - FR-3.3: BPSK modulation (logic 1 → -1.0, logic 0 → +1.0) - FR-3.4: I/Q sample generation at configurable rates - FR-3.5: Signal file export in multiple formats

FR-4: Message Decoding

The system shall decode received signals: - FR-4.1: Frame synchronization via sync pattern - FR-4.2: BCH(51,8) soft-decision decoding - FR-4.3: LDPC belief propagation decoding - FR-4.4: CRC validation - FR-4.5: Block deinterleaving

FR-5: Message Parsing

The system shall parse navigation messages: - FR-5.1: Extract FID and TOI from Subframe 1 - FR-5.2: Parse Clock & Ephemeris from Subframe 2 - FR-5.3: Route variable data from Subframe 3 - FR-5.4: Parse network access from Subframe 4 - FR-5.5: Calculate Time of Transmission

FR-6: Validation & Testing

The system shall provide validation capabilities: - FR-6.1: Generate test vectors for all components - FR-6.2: Validate against reference codes - FR-6.3: Measure BER vs SNR performance - FR-6.4: Verify LSIS compliance

Non-Functional Requirements

NFR-1: Performance

- NFR-1.1: Code generation < 1 second per PRN
- NFR-1.2: Frame encoding < 100ms per frame
- NFR-1.3: Frame decoding < 1 second per frame
- NFR-1.4: Real-time capable (process faster than signal duration)

NFR-2: Accuracy

- NFR-2.1: Spreading codes match Annex3 references exactly
- NFR-2.2: BER < 10^{-5} at SNR > 0 dB
- NFR-2.3: Frame synchronization > 99% reliability
- NFR-2.4: Time reconstruction accurate to code phase

NFR-3: Testability

- NFR-3.1: Unit tests for all components
- NFR-3.2: Integration tests for end-to-end flow
- NFR-3.3: Test coverage > 90%
- NFR-3.4: Reproducible test vectors

NFR-4: Usability

- NFR-4.1: Clear API documentation
- NFR-4.2: Working examples for all features
- NFR-4.3: Error messages with actionable guidance
- NFR-4.4: Configurable parameters

NFR-5: Maintainability

- NFR-5.1: Modular architecture
- NFR-5.2: Clear separation of concerns
- NFR-5.3: Consistent coding style
- NFR-5.4: Version control with Git

Technical Constraints

TC-1: Language & Tools

- Language choice is flexible (e.g., Python, Rust, C++, Go, Java)
- Use appropriate open source libraries for numerical operations and signal processing
- Standard library or equivalent for core algorithms
- Proprietary libraries discouraged for accessibility and reproducibility

TC-2: Data Formats

- Binary I/Q samples (int16, float32)
- CSV for analysis and debugging
- JSON for configuration
- HDF5 for large datasets (optional)

TC-3: Dependencies

- LDPC matrices from Annex1 CSV files
- Reference codes from Annex3 files
- LSIS specification document

TC-4: Compatibility

- Cross-platform (Windows, Linux, macOS)
- No RF hardware dependencies
- Standalone operation (no external services)
- Open source dependencies preferred

Success Metrics

1. **Completeness:** All 8 gateways delivered
2. **Correctness:** Pass all compliance tests
3. **Performance:** Meet all NFR-1 targets
4. **Quality:** Test coverage > 90%
5. **Usability:** Complete documentation and examples

Risk Assessment

Risk	Impact	Mitigation
LDPC decoder complexity	High	Use existing open source libraries or simplified algorithm
Test vector availability	Medium	Generate own test vectors from encoder
Performance bottlenecks	Medium	Profile and optimize critical paths
Specification ambiguity	Low	Reference NASA/ESA GNSS implementations
Time constraints	High	Prioritize core functionality, defer optional features